

Constraint Satisfaction Problems (CSP)

Lab instructor : Sir Saad Ahmed

Student : Mehr Ali

Roll no: 2P-0500

1.3 Problem 1: Map Coloring

```
In [1]: # Variables and their domains
variables = ["A", "B", "C", "D"]

domains = {
    "A": ["Red", "Green", "Blue"],
    "B": ["Red", "Green", "Blue"],
    "C": ["Red", "Green", "Blue"],
    "D": ["Red", "Green", "Blue"]
}

# Constraints: neighbors must have different colors
neighbors = [
    ("A", "B"),
    ("A", "C"),
    ("B", "C"),
    ("B", "D")
]
```

```
In [2]: def is_consistent(variable, value, assignment):
    for var1, var2 in neighbors:
        if variable == var1 and var2 in assignment and assignment[var2] == value:
            return False
        if variable == var2 and var1 in assignment and assignment[var1] == value:
            return False
    return True

def backtrack(assignment):
    if len(assignment) == len(variables):
        return assignment

    var = None
    for v in variables:
        if v not in assignment:
            var = v
```

```

        break

    for value in domains[var]:
        if is_consistent(var, value, assignment):
            assignment[var] = value
            result = backtrack(assignment)
            if result is not None:
                return result
            del assignment[var]

    return None

solution = backtrack({})
print(solution)

```

```
{'A': 'Red', 'B': 'Green', 'C': 'Blue', 'D': 'Red'}
```

```

In [3]: def backtrack_verbose(assignment):
        if len(assignment) == len(variables):
            return assignment

        var = None
        for v in variables:
            if v not in assignment:
                var = v
                break

        for value in domains[var]:
            if is_consistent(var, value, assignment):
                assignment[var] = value
                print("Assign:", var, "=", value, "| Current:", assignment)
                result = backtrack_verbose(assignment)
                if result is not None:
                    return result
                del assignment[var]
                print("Backtrack: undo", var, "=", value)

        return None

print("Solving map coloring:")
verbose_solution = backtrack_verbose({})
print("Solution:", verbose_solution)

```

Solving map coloring:

Assign: A = Red | Current: {'A': 'Red'}

Assign: B = Green | Current: {'A': 'Red', 'B': 'Green'}

Assign: C = Blue | Current: {'A': 'Red', 'B': 'Green', 'C': 'Blue'}

Assign: D = Red | Current: {'A': 'Red', 'B': 'Green', 'C': 'Blue', 'D': 'Red'}

Solution: {'A': 'Red', 'B': 'Green', 'C': 'Blue', 'D': 'Red'}

Backtracking Checkpoints

1. The algorithm does not need to backtrack in this ordering.
2. Total backtracks: 0.
3. It skips some combinations because backtracking stops as soon as a valid assignment is

found.

1.6 Problem 2: N-Queens

```
In [4]: N = 4

# Variables: one per column
variables = []
for i in range(N):
    variables.append(i)

# Domains: each queen can go in any row
domains = {}
for col in variables:
    rows = []
    for r in range(N):
        rows.append(r)
    domains[col] = rows
```

```
In [5]: def is_consistent(col, row, assignment):
    for other_col in assignment:
        other_row = assignment[other_col]
        if row == other_row:
            return False
        if abs(row - other_row) == abs(col - other_col):
            return False
    return True

def backtrack(assignment):
    if len(assignment) == len(variables):
        return assignment

    col = None
    for v in variables:
        if v not in assignment:
            col = v
            break

    for row in domains[col]:
        if is_consistent(col, row, assignment):
            assignment[col] = row
            result = backtrack(assignment)
            if result is not None:
                return result
            del assignment[col]

    return None

solution = backtrack({})
print(solution)

def print_board(solution, n):
```

```

    for row in range(n):
        line = ""
        for col in range(n):
            if solution[col] == row:
                line = line + " Q"
            else:
                line = line + " ."
        print(line)

print_board(solution, N)

```

```

{0: 1, 1: 3, 2: 0, 3: 2}
. . Q .
Q . . .
. . . Q
. Q . .

```

N-Queens Checkpoint

For $N = 4$, a valid solution is shown by the code. One example is {0: 1, 1: 3, 2: 0, 3: 2}.

Experiment & Explore

1. As N increases, the solver takes longer because there are more possible placements to check.
2. If we add region E as a neighbor of both C and D, it can still be solved with 3 colors.
3. If all 5 regions are neighbors of each other, we need 5 different colors.
4. For 4-Queens, there are 2 valid solutions.

In [6]: `import time`

```

def solve_n_queens(n):
    cols = []
    for i in range(n):
        cols.append(i)

    rows = {}
    for col in cols:
        row_list = []
        for r in range(n):
            row_list.append(r)
        rows[col] = row_list

    def is_safe(col, row, assignment):
        for other_col in assignment:
            other_row = assignment[other_col]
            if row == other_row:
                return False
            if abs(row - other_row) == abs(col - other_col):
                return False
        return True

```

```

def backtrack_queens(assignment):
    if len(assignment) == len(cols):
        return assignment
    col = None
    for v in cols:
        if v not in assignment:
            col = v
            break
    for row in rows[col]:
        if is_safe(col, row, assignment):
            assignment[col] = row
            result = backtrack_queens(assignment)
            if result is not None:
                return result
            del assignment[col]
    return None

return backtrack_queens({})

print("N | Solution Found? | Approx. Time")
for n in [4, 8, 10, 12]:
    start = time.perf_counter()
    result = solve_n_queens(n)
    end = time.perf_counter()
    found = result is not None
    print(n, "|", found, "|", round(end - start, 6), "seconds")

```

```

N | Solution Found? | Approx. Time
4 | True | 6.6e-05 seconds
8 | True | 0.001445 seconds
10 | True | 0.001315 seconds
12 | True | 0.004542 seconds

```

```

In [7]: def solve_map_coloring(region_list, color_domains, constraint_pairs):
def is_safe(region, color, assignment):
    for first, second in constraint_pairs:
        if region == first and second in assignment and assignment[second] == color:
            return False
        if region == second and first in assignment and assignment[first] == color:
            return False
    return True

def backtrack_map(assignment):
    if len(assignment) == len(region_list):
        return assignment
    region = None
    for item in region_list:
        if item not in assignment:
            region = item
            break
    for color in color_domains[region]:
        if is_safe(region, color, assignment):
            assignment[region] = color
            result = backtrack_map(assignment)
            if result is not None:
                return result

```

```

        del assignment[region]
        return None

    return backtrack_map({})

regions = ["A", "B", "C", "D", "E"]
three_color_domains = {
    "A": ["Red", "Green", "Blue"],
    "B": ["Red", "Green", "Blue"],
    "C": ["Red", "Green", "Blue"],
    "D": ["Red", "Green", "Blue"],
    "E": ["Red", "Green", "Blue"]
}
neighbors_e = [("A", "B"), ("A", "C"), ("B", "C"), ("B", "D"), ("C", "E"), ("D", "E")]
print("E map with 3 colors:", solve_map_coloring(regions, three_color_domains, neighbors_e))

full_neighbors = []
for i in range(len(regions)):
    for j in range(i + 1, len(regions)):
        full_neighbors.append((regions[i], regions[j]))

four_color_domains = {
    "A": ["Red", "Green", "Blue", "Yellow"],
    "B": ["Red", "Green", "Blue", "Yellow"],
    "C": ["Red", "Green", "Blue", "Yellow"],
    "D": ["Red", "Green", "Blue", "Yellow"],
    "E": ["Red", "Green", "Blue", "Yellow"]
}
five_color_domains = {
    "A": ["Red", "Green", "Blue", "Yellow", "Purple"],
    "B": ["Red", "Green", "Blue", "Yellow", "Purple"],
    "C": ["Red", "Green", "Blue", "Yellow", "Purple"],
    "D": ["Red", "Green", "Blue", "Yellow", "Purple"],
    "E": ["Red", "Green", "Blue", "Yellow", "Purple"]
}
print("Full map with 4 colors:", solve_map_coloring(regions, four_color_domains, full_neighbors))
print("Full map with 5 colors:", solve_map_coloring(regions, five_color_domains, full_neighbors))

```

E map with 3 colors: {'A': 'Red', 'B': 'Green', 'C': 'Blue', 'D': 'Red', 'E': 'Green'}

Full map with 4 colors: None

Full map with 5 colors: {'A': 'Red', 'B': 'Green', 'C': 'Blue', 'D': 'Yellow', 'E': 'Purple'}

```

In [8]: def count_all_n_queens_solutions(n):
        cols = []
        for i in range(n):
            cols.append(i)

        rows = {}
        for col in cols:
            row_list = []
            for r in range(n):
                row_list.append(r)
            rows[col] = row_list

```

```

def is_safe(col, row, assignment):
    for other_col in assignment:
        other_row = assignment[other_col]
        if row == other_row:
            return False
        if abs(row - other_row) == abs(col - other_col):
            return False
    return True

def backtrack_all(assignment):
    if len(assignment) == len(cols):
        return 1
    col = None
    for v in cols:
        if v not in assignment:
            col = v
            break
    count = 0
    for row in rows[col]:
        if is_safe(col, row, assignment):
            assignment[col] = row
            count = count + backtrack_all(assignment)
            del assignment[col]
    return count

return backtrack_all({})

print("Total valid solutions for 4-Queens:", count_all_n_queens_solutions(4))

```

Total valid solutions for 4-Queens: 2

1.8 Challenge: Sudoku Solver

```

In [9]: board = [
    [5, 3, 0, 0, 7, 0, 0, 0, 0],
    [6, 0, 0, 1, 9, 5, 0, 0, 0],
    [0, 9, 8, 0, 0, 0, 0, 6, 0],
    [8, 0, 0, 0, 6, 0, 0, 0, 3],
    [4, 0, 0, 8, 0, 3, 0, 0, 1],
    [7, 0, 0, 0, 2, 0, 0, 0, 6],
    [0, 6, 0, 0, 0, 0, 2, 8, 0],
    [0, 0, 0, 4, 1, 9, 0, 0, 5],
    [0, 0, 0, 0, 8, 0, 0, 7, 9]
]

def find_empty(board):
    for row in range(9):
        for col in range(9):
            if board[row][col] == 0:
                return (row, col)
    return None

def is_valid(board, row, col, num):

```

```
    for i in range(9):
        if board[row][i] == num:
            return False
        if board[i][col] == num:
            return False

    box_row = (row // 3) * 3
    box_col = (col // 3) * 3
    for i in range(box_row, box_row + 3):
        for j in range(box_col, box_col + 3):
            if board[i][j] == num:
                return False

    return True

def solve(board):
    empty = find_empty(board)
    if empty is None:
        return True

    row, col = empty

    for num in range(1, 10):
        if is_valid(board, row, col, num):
            board[row][col] = num
            if solve(board):
                return True
            board[row][col] = 0

    return False

def print_board(board):
    for row in range(9):
        line = ""
        for col in range(9):
            line = line + str(board[row][col]) + " "
        print(line)

if solve(board):
    print("Solved!")
    print_board(board)
else:
    print("No solution exists.")
```

Solved!

```
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9
```



```
In [10]: def verify_board(board):
    digits = set(range(1, 10))

    for row in range(9):
        if set(board[row]) != digits:
            return False

    for col in range(9):
        column_values = set()
        for row in range(9):
            column_values.add(board[row][col])
        if column_values != digits:
            return False

    for box_row in range(0, 9, 3):
        for box_col in range(0, 9, 3):
            box_values = set()
            for row in range(box_row, box_row + 3):
                for col in range(box_col, box_col + 3):
                    box_values.add(board[row][col])
            if box_values != digits:
                return False

    return True

print("Verification:", verify_board(board))
```

Verification: True

```
In [11]: hard_board = [
    [8, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 3, 6, 0, 0, 0, 0, 0],
    [0, 7, 0, 0, 9, 0, 2, 0, 0],
    [0, 5, 0, 0, 0, 7, 0, 0, 0],
    [0, 0, 0, 0, 4, 5, 7, 0, 0],
    [0, 0, 0, 1, 0, 0, 0, 3, 0],
    [0, 0, 1, 0, 0, 0, 0, 6, 8],
    [0, 0, 8, 5, 0, 0, 0, 1, 0],
    [0, 9, 0, 0, 0, 0, 4, 0, 0]
]

if solve(hard_board):
    print("Hard Sudoku solved!")
    print_board(hard_board)
else:
    print("No solution exists.")
```

Hard Sudoku solved!

```
8 1 2 7 5 3 6 4 9
9 4 3 6 8 2 1 7 5
6 7 5 4 9 1 2 8 3
1 5 4 2 3 7 8 9 6
3 6 9 8 4 5 7 2 1
2 8 7 1 6 9 5 3 4
5 2 1 9 7 4 3 6 8
4 3 8 5 2 6 9 1 7
7 9 6 3 1 8 4 5 2
```

Sudoku Short Answers

The `is_valid` function checks the row, column, and 3x3 box. The `solve` function uses backtracking by trying numbers 1 to 9, and undoing a choice if it does not work.

1.9 Reflection

1. Backtracking means trying one choice at a time and returning to the previous step if that choice causes a conflict. It is more efficient than checking every full combination because it stops early when a partial assignment is already impossible.
2. If every region is a neighbor of every other region in the original 4-region map, we need 4 colors. Each region must have a different color because no two neighboring regions can share the same one.
3. Representing a problem as a CSP helps because we can describe it using variables, domains, and constraints instead of writing a special algorithm from scratch. It gives us one general way to solve many different problems with backtracking.
4. If we did not check constraints in N-Queens, the solver would place queens anywhere and most boards would be invalid. For $N = 8$, it would need to check 8^8 combinations, which is a lot more work.